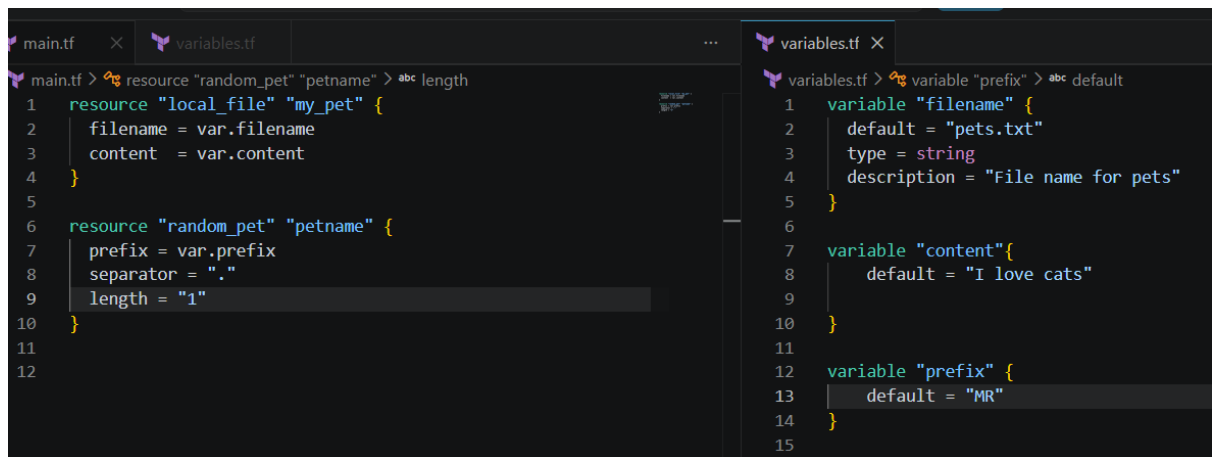


Terraform-03

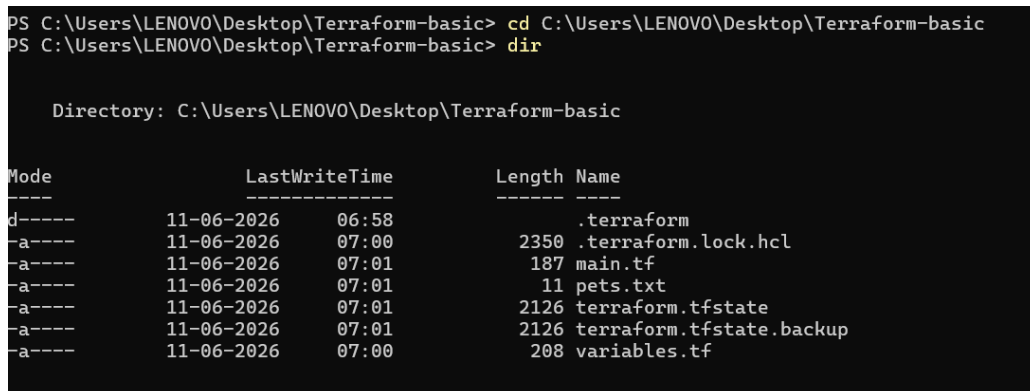
1. Watch the Terraform-03 video.

- Using of variables: By using variables.tf file

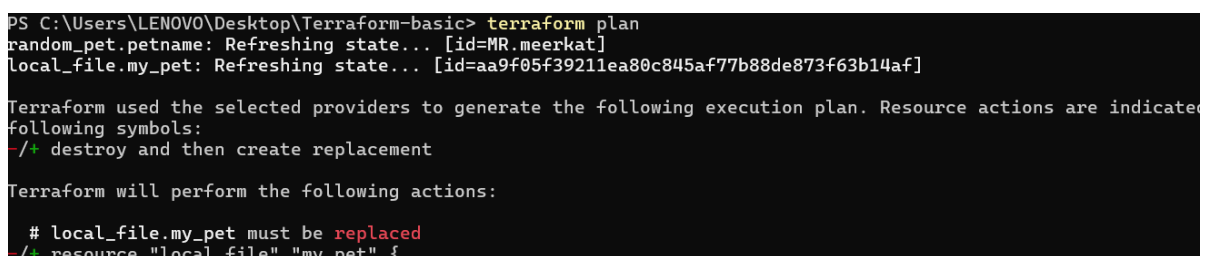


The screenshot shows a code editor with two files open. The left file, `main.tf`, contains two resource blocks. The first block, `resource "local_file" "my_pet"`, uses variables `var.filename` and `var.content`. The second block, `resource "random_pet" "petname"`, uses variables `var.prefix` and `var.separator`, and sets `length = "1"`. The right file, `variables.tf`, defines three variables: `filename` with a default of `"pets.txt"`, `content` with a default of `"I love cats"`, and `prefix` with a default of `"MR"`.

- Create dir wild.txt: By using interactive mode



The terminal shows the current directory `C:\Users\LENOVO\Desktop\Terraform-basic` and its contents. The directory listing includes files like `.terraform`, `.terraform.lock.hcl`, `main.tf`, `pets.txt`, `terraform.tfstate`, `terraform.tfstate.backup`, and `variables.tf`. Below this, the `terraform plan` command is executed, showing the state of the resources and the actions Terraform will perform.



The terminal shows the output of the `terraform plan` command. It indicates that the state is being refreshed for `random_pet.petname` and `local_file.my_pet`. The plan shows that `local_file.my_pet` must be replaced. The actions Terraform will perform are listed below.

Plan: 1 to add, 0 to change, 1 to destroy.

Note: You didn't use the `-out` option to save this plan, so Terraform can't guarantee to take exactly what you run `"terraform apply"` now.

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
local_file.my_pet: Refreshing state... [id=aa9f05f39211ea80c845af77b88de873f63b14af]
random_pet.petname: Refreshing state... [id=MR.meerkat]

```

Enter a value: yes
local_file.my_pet: Destroying... [id=aa9f05f39211ea80c845af77b88de873f63b14af]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=aa9f05f39211ea80c845af77b88de873f63b14af]
Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

```

After terraform init, plan and apply then we get new wild.txt

```

() terraform.tfstate
  terraform.tfstate.backup
  variables.tf
  wild.txt

```

```

main.tf  wild.txt
1  I love cats

```

➤ Command line flags

```

main.tf  wild.txt  variables.tf
main.tf > ...
1  resource "local_file" "my_pet" {
2    filename = var.filename
3    content  = var.content
4  }
5
6  resource "random_pet" "petname" {
7    prefix = var.prefix
8    separator = "."
9    length = "1"
10 }
11

variables.tf > variable "prefix"
1  variable "filename" {
2
3  }
4
5  variable "content" {
6
7  }
8
9
10 variable "prefix" {
11
12 }
13

```

```

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
var.content
Enter a value: yes

var.prefix
Enter a value: no

random_pet.petname: Refreshing state... [id=MR.meerkat]
local_file.my_pet: Refreshing state... [id=aa9f05f39211ea80c845af77b88de873f63b14af]

```

```
# random_pet.petname must be replaced
/+ resource "random_pet" "petname" {
  ~ id          = "MR.meerkat" -> (known after apply)
  ~ prefix      = "MR" -> "no" # forces replacement
    # (2 unchanged attributes hidden)
}
```

Plan: 2 to add, 0 to change, 2 to destroy.

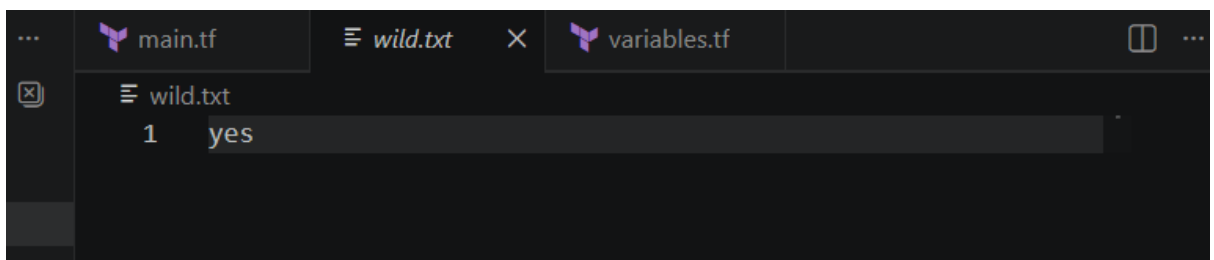
Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

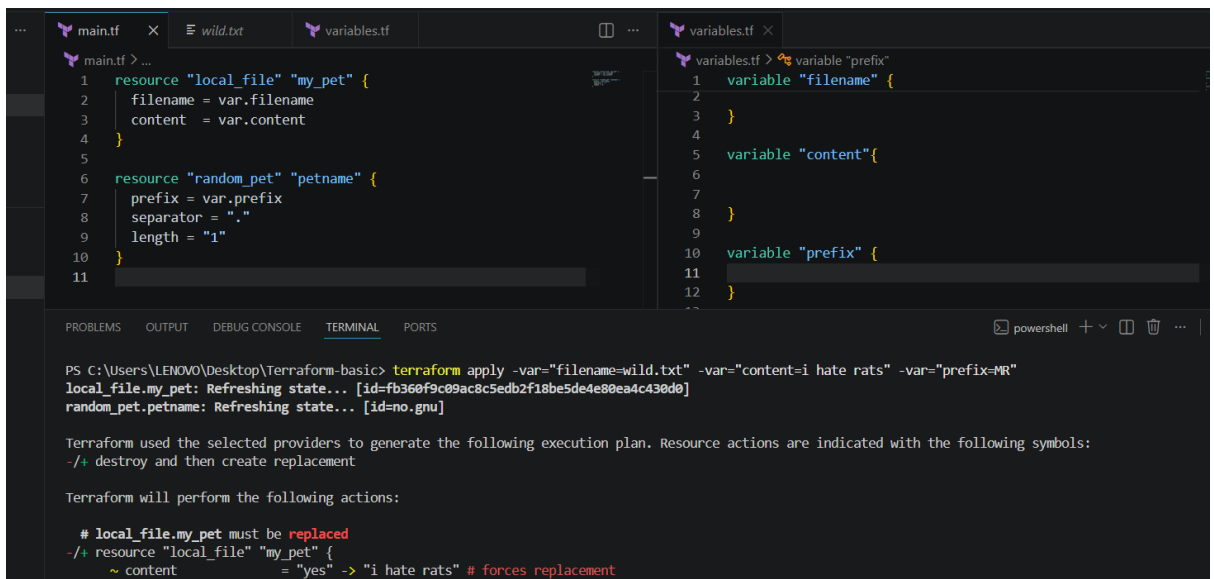
Enter a value: yes

```
random_pet.petname: Creating...
local_file.my_pet: Creating...
random_pet.petname: Creation complete after 0s [id=no.gnu]
local_file.my_pet: Creation complete after 0s [id=fb360f9c09ac8c5edb2f18be5de4e80ea4c430d0]

Apply complete! Resources: 2 added, 0 changed, 2 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform-basic>
```



Command line flags: terraform apply -var "filename=/root/pets.txt" -var "prefix=MR"



```
main.tf  wild.txt  variables.tf
wild.txt
1 i hate rats
```

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> Set-Item -path env:TF_VAR_filename -Value 'wild.txt'
PS C:\Users\LENOVO\Desktop\Terraform-basic> Set-Item -path env:TF_VAR_filename -Value 'I love Horses'
PS C:\Users\LENOVO\Desktop\Terraform-basic> Set-Item -path env:TF_VAR_filename -Value 'MISS'
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
var.content
  Enter a value: Horses

var.prefix
  Enter a value: MISS

random_pet.petname: Refreshing state... [id=No.woodcock]
local_file.my_pet: Refreshing state... [id=fb360f9c09ac8c5edb2f18be5de4e80ea4c430d0]
```

```
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

random_pet.petname: Destroying... [id=No.woodcock]
local_file.my_pet: Destroying... [id=fb360f9c09ac8c5edb2f18be5de4e80ea4c430d0]
random_pet.petname: Destruction complete after 0s
local_file.my_pet: Destruction complete after 0s
random_pet.petname: Creating...
random_pet.petname: Creation complete after 0s [id=MISS.terrapin]
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=29ed16888c8beb19c7a0740a95b969d2562314a4]

Apply complete! Resources: 2 added, 0 changed, 2 destroyed.
```

- Environment variables
export TF_VAR_filename="/root.pets.txt"
export TF_VAR_prefix= "MR"
Set-Item -Path env:TF_VAR_filename -Value 'wild.txt'

```
File Edit Selection View Go Run Terminal Help
Terraform-basic
EXPLORER
  OPEN EDITORS
    GROUP 1
      main.tf
      variables.tf
    GROUP 2
      variables.tf
      CLF.txt
  TERRAFORM-BASIC
    .terraform
    .terraform.lock.hcl
    CLF.txt
    main.tf
    terraform.tfstate
    terraform.tfstate.backup
    variables.tf
main.tf  variables.tf  CLF.txt
main.tf
5 resource "random_pet" "petname"
6
7 resource "random_pet" "petname" {
8   prefix = var.prefix
9   separator = "."
10  length = "1"
11 }
variables.tf
1 CLF
CLF.txt
1 CLF
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\LENOVO\Desktop\Terraform-basic> Set-Item -path env:TF_VAR_filename -Value 'MISS'
PS C:\Users\LENOVO\Desktop\Terraform-basic> Set-Item -path env:TF_VAR_filename -Value 'I love Horses'
PS C:\Users\LENOVO\Desktop\Terraform-basic> Set-Item -path env:TF_VAR_filename -Value 'wild.txt'
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply -var="filename=CLF.txt" -var="content=CLF" -var="prefix=MR"
local_file.my_pet: Refreshing state... [id=fb360f9c09ac8c5edb2f18be5de4e80ea4c430d0]
random_pet.petname: Refreshing state... [id=no.lamb]
```

```

Plan: 2 to add, 0 to change, 2 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my_pet: Destroying... [id=fb360f9c09ac8c5edb2f18be5de4e80ea4c430d0]
random_pet.petname: Destroying... [id=no.lamb]
random_pet.petname: Destruction complete after 0s
local_file.my_pet: Destruction complete after 0s
random_pet.petname: Creating...
random_pet.petname: Creation complete after 0s [id=MR.escargot]
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=df0d2b6b2145603bd1f3b4]

Apply complete! Resources: 2 added, 0 changed, 2 destroyed.

```

➤ Terraform apply next

The screenshot shows the VS Code interface with the following components:

- EXPLORER:**
 - OPEN EDITORS: main.tf, variables.tf
 - GROUP 1: main.tf, variables.tf
 - GROUP 2: variables.tf, CTF.txt
 - TERRAFORM-BASIC:
 - .terraform
 - .terraform.lock.hcl
 - main.tf
 - pets.txt
 - terraform.tfstate
 - terraform.tfstate.backup
 - variables.tf
- EDITOR:**
 - main.tf:


```

5 resource "random_pet" "petname" {
6   prefix = var.prefix
7   separator = "."
8   length = "1"
9 }
10
11

```
 - variables.tf:


```

5 variable "prefix" {
6   default = "Hello World"
7 }
8
9
10 variable "content" {
11   default = "MR"
12 }
13
14

```
- TERMINAL:**

```

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
random_pet.petname: Refreshing state... [id=MR.ferret]
local_file.my_pet: Refreshing state... [id=df0d2b6b2145603bd1f3b4]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  ~ destroy and then create replacement

Terraform will perform the following actions:

# local_file.my_pet must be replaced
~ resource "local_file" "my_pet" {
  ~ content
  ~ filename
  ~ filename
  ~ content
}

```

```

Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my_pet: Destroying... [id=df0d2b6b2145603bd1f3b4]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=0a4d55a8d778e5022fab701977c5d840bbc486d0]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform-basic>

```

2. Execute the script shown in the video.

➤ Close vs code and open it again so environment variables close and apply now

The screenshot shows the VS Code interface with the following components:

- EXPLORER:**
 - OPEN EDITORS: main.tf
 - TERRAFORM-BASIC:
 - .terraform
 - .terraform.lock.hcl
 - main.tf
 - pets.txt
 - terraform.tfstate
 - terraform.tfstate.backup
- EDITOR:**
 - main.tf:


```

1 resource "local_file" "my_pet" {
2   filename = "pets.txt"
3   content = "My cat name is ${random_pet.petname.id}"
4   depends_on = [
5     random_pet.petname
6   ]
7 }
8
9
10 resource "random_pet" "petname" {
11   prefix = "MR"
12   separator = "."
13   length = "1"
14 }
15

```
- TERMINAL:**

```

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
random_pet.petname: Refreshing state... [id=MR.ferret]
local_file.my_pet: Refreshing state... [id=0a4d55a8d778e5022fab701977c5d840bbc486d0]

```

```

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.my_pet: Destroying... [id=0a4d55a8d778e5022fab701977c5d840bbc486d0]
local_file.my_pet: Destruction complete after 0s
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=ba7d8973c074119188b5c1da0537e31a99b829cf]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\Users\LENOVO\Desktop\Terraform-basic>

```

➤ Now check output of terraform after apply

The screenshot shows a Visual Studio Code editor with a Terraform configuration file named `main.tf`. The configuration defines a `random_pet` resource with a `petname` attribute and an `output` named `pet_name` that references the `id` of the `random_pet` resource. Below the editor, the `TERMINAL` tab is active, showing the output of the `terraform output` command. The output displays a warning that no outputs were found in the state file. Subsequently, the `terraform apply` command is executed, which refreshes the state for both the `random_pet` and `local_file.my_pet` resources. The final output shows that the `pet_name` output is now set to `"MR.ferret"`.

```

main.tf > ...
10 resource "random_pet" "petname" {
11     separator = "."
12     length = "1"
13 }
14
15
16 output "pet_name" {
17     value = random_pet.petname.id
18 }
19
20

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform output

Warning: No outputs found

The state file either has no outputs defined, or all the defined outputs are empty. Please de
and run `terraform refresh` for it to become available. If you are using interpolation, pleas
`terraform console` command to assist.

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform apply
random_pet.petname: Refreshing state... [id=MR.ferret]
local_file.my_pet: Refreshing state... [id=ba7d8973c074119188b5c1da0537e31a99b829cf]

Changes to Outputs:
+ pet_name = "MR.ferret"

```

```

Enter a value: yes

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

pet_name = "MR.ferret"
PS C:\Users\LENOVO\Desktop\Terraform-basic>

```

•

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform output
pet_name = "MR.ferret"
PS C:\Users\LENOVO\Desktop\Terraform-basic> []
```

➤ Terraform show

```
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform output
pet_name = "MR.ferret"
PS C:\Users\LENOVO\Desktop\Terraform-basic> terraform show
# local_file.my_pet:
resource "local_file" "my_pet" {
  content          = "My cat name is MR.ferret"
  content_base64sha256 = "wwFqffNOOLl0M0eRd77MAMjQS9BoyTV9OQxNTQlEJro="
  content_base64sha512 = "DjX1NG9CEMuwJRroC0SfzWW04gWL8yf7toRu/jY7qK1IiQ5mU5iccVy/kndIKKJwbZ9PeT89E
  content_md5       = "0b3916d81ff041be1b82180596d0f99c"
  content_sha1      = "ba7d8973c074119188b5c1da0537e31a99b829cf"
  content_sha256    = "c3016a7c534e38b97433479177becc0268d04bd068cad57d390c4d4d094426ba"
  content_sha512    = "0e35f5346f4210cbb0251ae80b449fcd65b4e2058bf327fbb6846efe363ba8ad488a243999
  directory_permission = "0777"
  file_permission     = "0777"
  filename            = "pets.txt"
  id                  = "ba7d8973c074119188b5c1da0537e31a99b829cf"
```

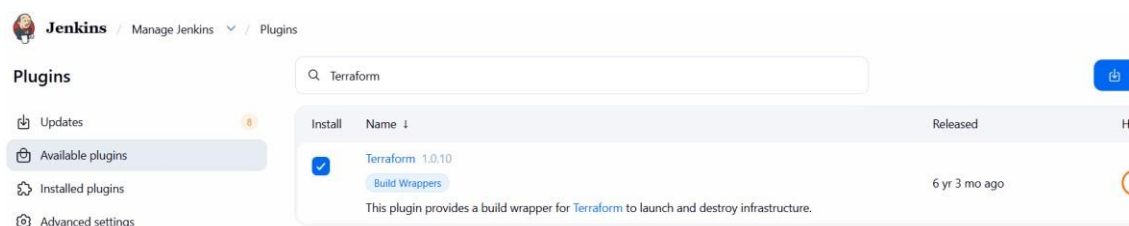
3. Integrate Terraform in Jenkins using the Terraform plugin.

Open Jenkins server and check running or not and version also

```
root@ip-172-31-22-87:~# systemctl start jenkins
root@ip-172-31-22-87:~# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-06-09 17:22:57 UTC; 37s ago
     Main PID: 519 (java)
       Tasks: 45 (limit: 1013)
      Memory: 389.3M (peak: 390.6M)
         CPU: 22.347s
        CGroup: /system.slice/jenkins.service
                └─519 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.wa
```

```
root@ip-172-31-22-87:~# terraform -v
Terraform v1.15.5
on linux_amd64
root@ip-172-31-22-87:~# []
```

Open Jenkins in browser and install Terraform plugin



In Jenkins manage select Tools and add Terraform installations details

Jenkins / Manage Jenkins / Tools

Terraform installations

+ Add Terraform

Terraform

Name

terraform

Install directory

/usr/bin

Now select new job item and select pipeline

Jenkins / All / New Item

New Item

Enter an item name

Sample Terraform

Select an item type

Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple

Now in Jenkins server go to workspace of new created job and there create main.tf

```
root@ip-172-31-22-87:/var/lib/jenkins/workspace# cd ..
root@ip-172-31-22-87:/var/lib/jenkins# cd workspace/Sample\ Terraform
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform# vi main.tf
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform# ls
main.tf
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform#
```

Now in main.tf file write script and save it

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

resource "local_file" "demo" {
  filename = "terraform-from-jenkins.txt"
  content  = "Terraform executed successfully from Jenkins using Terraform Plugin"
}
```


Now Pipeline and write a declarative script for the new created job

 **Jenkins** / Sample Terraform / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1 pipeline {  
2   agent any  
3  
4   stages {  
5  
6     stage('Terraform Version') {  
7       steps {  
8         sh 'terraform -v'  
9       }  
10    }  
}
```

Pipeline Script:

```
pipeline {  
  agent any  
  
  stages {  
  
    stage('Terraform Version') {  
      steps {  
        sh 'terraform -v'  
      }  
    }  
  
    stage('Terraform  
Init') {  
      steps {  
        sh 'terraform init'  
      }  
    }  
  
    stage('Terraform Plan') {  
      steps {  
        sh 'terraform plan'  
      }  
    }  
  
    stage('Terraform Apply') {  
      steps {  
        sh 'terraform apply -auto-approve'  
      }  
    }  
  }  
}
```

Successful script

ns / Sample Terraform ▾ / #1 ▾

```
[32m+0[0m[0m content_base64sha256 = (known after apply)
[32m+0[0m[0m content_base64sha512 = (known after apply)
[32m+0[0m[0m content_md5          = (known after apply)
[32m+0[0m[0m content_sha1          = (known after apply)
[32m+0[0m[0m content_sha256        = (known after apply)
[32m+0[0m[0m content_sha512         = (known after apply)
[32m+0[0m[0m directory_permission = "0777"
[32m+0[0m[0m file_permission      = "0777"
[32m+0[0m[0m filename             = "terraform-from-jenkins.txt"
[32m+0[0m[0m id                  = (known after apply)
}

[1mPlan:[0m [0m1 to add, 0 to change, 0 to destroy.
[0m[1mlocal_file.demo: Creating...[0m[0m
[0m[1mlocal_file.demo: Creation complete after 0s [id=221065dac85086259e58924cd5d80a552d879e04]
[0m[1m[32m
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.[0m
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Now check in server successful

```
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform# terraform apply -auto-approve
local_file.demo: Refreshing state... [id=221065dac85086259e58924cd5d80a552d879e04]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform#
```

To check open Vi terraform-from-jenkins.txt

```
-rw-r--r-- 1 jenkins jenkins 67 Jun 10 09:09 terraform-from-jenkins.txt
-rw-r--r-- 1 jenkins jenkins 1694 Jun 10 09:09 terraform.tfstate
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform# vi terraform-from-jenkins.txt
root@ip-172-31-22-87:/var/lib/jenkins/workspace/Sample Terraform#
```

Successfully from Jenkins using Terraform plugin

4. Create one Jenkins job using **Maven Project** for the code below with **two stages**:

- **Stage 1:** Git clone
- **Stage 2:** Maven Compilation
- **Code:** <https://github.com/betawins/java-Working-app.git>

➤ Connect with Jenkins server and open in browser and create a new job

New Item

Enter an item name

Hiring-App

Select an item type



Pipeline

Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

➤ Select Source code as git and cloning URL id paste it

Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git ?

Repositories ?

Repository URL ?

https://github.com/Arunsai873/hiring-app.git

Credentials ?

- none -

➤ Branch set as main and save it

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

➤ Maven integration plugin installed

Plugins

Updates

11

Available plugins

Installed plugins

Advanced settings

Q mave

Name ↓

Health

Enabled

[Maven Integration plugin 3.27](#)

This plugin provides a deep integration between Jenkins and Maven. It adds support for automatic triggers between projects depending on SNAPSHOTS as well as the automated configuration of various Jenkins publishers such as Junit.

[Report an issue with this plugin](#)

95



- In Manage Jenkins open and in tools select maven and select instant automatically

Maven installations ^  Edited

+ Add Maven

⋮ Maven

Name

Maven

☒ Install automatically ?

⋮ Install from Apache

Version

3.9.16

+ Add Installer

- In Build steps select maven and set goals and save it and build now

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment

⋮ Invoke top-level Maven targets ?

Maven Version

Maven

Goals

clean compile

- Successfully Build

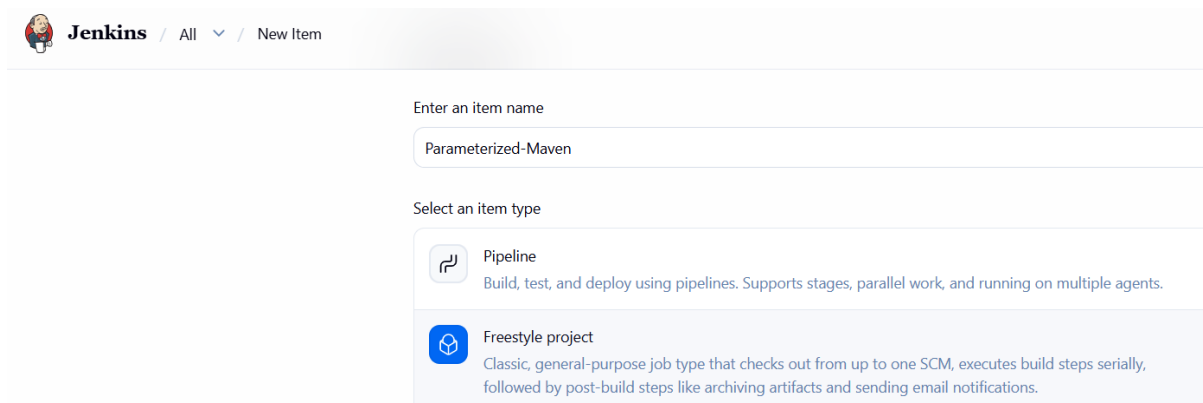
```
Progress (1): 11/30 kB
Progress (1): 28/30 kB
Progress (1): 30 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-c
2.16.2.jar (30 kB at 490 kB/s)
[INFO] No sources to compile
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 5.035 s
[INFO] Finished at: 2026-06-11T09:54:28Z
[INFO] -----
Finished: SUCCESS
```

5. Use the same code and create a **parameterized job** in Jenkins with:

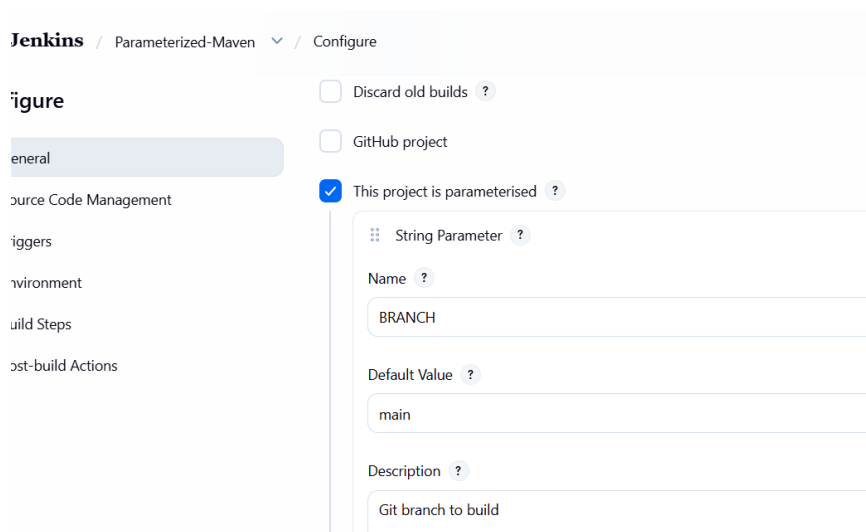
- **Stage 1:** Git clone
- **Stage 2:** Maven Compilation
- **Code:** <https://github.com/betawins/java-Working-app.git>

➤ Open Jenkins browsers and select new job and create with freestyle



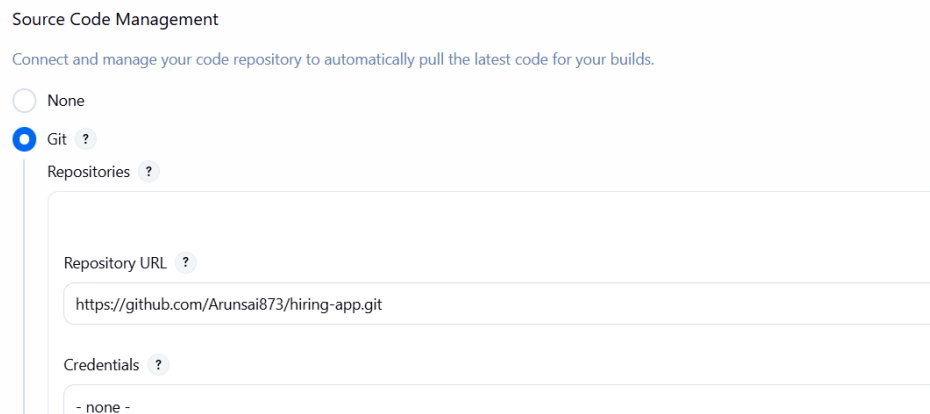
The screenshot shows the Jenkins 'New Item' page. At the top, there's a breadcrumb 'Jenkins / All / New Item'. Below it, a text input field 'Enter an item name' contains 'Parameterized-Maven'. Underneath, a section 'Select an item type' shows two options: 'Pipeline' (with a blue icon) and 'Freestyle project' (with a blue icon). The 'Freestyle project' option is highlighted with a light blue background. Its description reads: 'Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.'

➤ Select Parameterized and select string parameter



The screenshot shows the 'Configure' page for the 'Parameterized-Maven' job. On the left, a sidebar lists various configuration sections: 'General', 'Source Code Management', 'Triggers', 'Environment', 'Build Steps', and 'Post-build Actions'. The 'General' section is selected. In the main area, there are three checkboxes: 'Discard old builds', 'GitHub project', and 'This project is parameterised'. The 'This project is parameterised' checkbox is checked. Below it, a 'String Parameter' is configured with the name 'BRANCH' and the default value 'main'. The description field contains the text 'Git branch to build'.

➤ Select Git and given Git URL and select branch main



The screenshot shows the 'Source Code Management' configuration section. It starts with the instruction 'Connect and manage your code repository to automatically pull the latest code for your builds.' There are two radio buttons: 'None' and 'Git'. The 'Git' radio button is selected. Below it, a 'Repositories' section contains a 'Repository URL' field with the value 'https://github.com/Arunsai873/hiring-app.git' and a 'Credentials' dropdown menu showing '- none -'.

•

- Now select maven build and save it

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment

⋮

Invoke top-level Maven targets ?

Maven Version

Maven

Goals

clean compile

- Successfully Build

```
[INFO]
[INFO] --- compiler:3.15.0:compile (default-compile) @ hiring ---
[INFO] No sources to compile
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.343 s
[INFO] Finished at: 2026-06-11T10:07:40Z
[INFO] -----
Finished: SUCCESS
```

6. What are the **global variables** in Jenkins?

Global variables in Jenkins are predefined environment variables accessible across jobs and pipelines. They provide build-related information such as build number, workspace path, job name, Git branch, and Jenkins home directory.

Some commonly used global variables include

BUILD_NUMBER, which stores the current build number,

JOB_NAME, which contains the name of the Jenkins job, and

WORKSPACE, which defines the location where Jenkins clones the source code and executes builds.

Variables like GIT_COMMIT and GIT_BRANCH help identify the exact Git commit and branch used during the build process.

These global variables are useful in automation, scripting, and CI/CD pipelines to make Jenkins jobs more dynamic and reusable.

•

Common Global Variables

Variable	Purpose
BUILD_NUMBER	Current build number
BUILD_ID	Build ID
JOB_NAME	Name of Jenkins job
BUILD_URL	URL of current build
WORKSPACE	Workspace location
JENKINS_URL	Jenkins server URL
NODE_NAME	Jenkins agent name
BUILD_TAG	Unique build tag
GIT_BRANCH	Current Git branch
GIT_COMMIT	Commit ID
EXECUTOR_NUMBER	Executor number
HOME	Jenkins home directory